Polyfills, Utility Functions, and Common Implementations

This file covers commonly asked interview implementations: debounce, throttle, polyfills, and utilities.

1) Throttle and Debounce Implementations

1.1 Debounce (Delay Action Until Idle)

```
function debounce(fn, delay) {
  let timeout;

  return function debounced(...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// Use case: search input
const handleSearch = debounce((query) => {
  console.log("searching:", query);
  fetch(`/api/search?q=${query}`);
}, 300);

input.addEventListener("input", (e) => {
  handleSearch(e.target.value);
});
// only searches 300ms after user stops typing
```

1.2 Throttle (Max Frequency)

```
function throttle(fn, delay) {
  let lastRun = 0;
  let timeout;

  return function throttled(...args) {
    const now = Date.now();
    const remaining = delay - (now - lastRun);

    if (remaining <= 0) {
      fn.apply(this, args);
      lastRun = now;
    } else {
      clearTimeout(timeout);
      timeout = setTimeout(() => {
        fn.apply(this, args);
```

```
        lastRun = Date.now();
      }, remaining);
    }
  };
}

// Use case: window scroll
window.addEventListener("scroll", throttle(() => {
  console.log("scroll event");
  updateScrollPosition();
}, 100));
// fires at most once per 100ms
```

1.3 Difference

- Debounce: waits for silence, then fires once.
- Throttle: fires at regular intervals.

2) Memoization Polyfill

2.1 Simple Memoize

```
function memoize(fn, options = {}) {
  const cache = new Map();
  const { maxSize = 100 } = options;

  return function memoized(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      return cache.get(key);
    }

    const result = fn.apply(this, args);

    if (cache.size >= maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
    }

    cache.set(key, result);
    return result;
  };
}

const expensive = memoize((n) => {
  console.log("computing fibonacci...");
  return fibonacci(n);
}, { maxSize: 10 });
```

```
console.log(expensive(5)); // computing fibonacci...
console.log(expensive(5)); // (cached)
```

2.2 WeakMap Memoize for Objects

```
function memoizeWithWeakMap(fn) {
  const cache = new WeakMap();

  return function (obj) {
    if (cache.has(obj)) {
      return cache.get(obj);
    }

    const result = fn(obj);
    cache.set(obj, result);
    return result;
  };
}

const getUserName = memoizeWithWeakMap((user) => {
  return user.name.toUpperCase();
});

const user = { name: "alice" };
console.log(getUserName(user)); // ALICE
console.log(getUserName(user)); // ALICE (cached)
```

3) Promise Polyfills

3.1 Promise.all Implementation

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (promises.length === 0) {
      resolve([]);
      return;
    }

    const results = [];
    let completed = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then((value) => {
          results[index] = value;
          completed++;
          if (completed === promises.length) {
            resolve(results);
          }
```

```
        })
        .catch(reject);
    });
  });
}

promiseAll([
  Promise.resolve(1),
  Promise.resolve(2),
  new Promise(r => setTimeout(() => r(3), 100))
]).then(console.log); // [1, 2, 3]
```

3.2 Promise.race Implementation

```
function promiseRace(promises) {
  return new Promise((resolve, reject) => {
    promises.forEach((promise) => {
      Promise.resolve(promise)
        .then(resolve)
        .catch(reject);
    });
  });
}
```

3.3 Promise.allSettled Implementation

```
function promiseAllSettled(promises) {
  return Promise.all(
    promises.map(p =>
      Promise.resolve(p)
        .then(value => ({ status: "fulfilled", value }))
        .catch(reason => ({ status: "rejected", reason }))
    )
  );
}
```

4) Array Polyfills

4.1 Array.prototype.map

```
Array.prototype.customMap = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    result.push(callback.call(thisArg, this[i], i, this));
  }
  return result;
};
```

```
const arr = [1, 2, 3];
console.log(arr.customMap(x => x * 2)); // [2, 4, 6]
```

4.2 Array.prototype.filter

```
Array.prototype.customFilter = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (callback.call(thisArg, this[i], i, this)) {
      result.push(this[i]);
    }
  }
  return result;
};

const arr = [1, 2, 3, 4, 5];
console.log(arr.customFilter(x => x > 2)); // [3, 4, 5]
```

4.3 Array.prototype.reduce

```
Array.prototype.customReduce = function(callback, initialValue) {
  let accumulator = initialValue;
  let startIndex = 0;

  if (initialValue === undefined) {
    accumulator = this[0];
    startIndex = 1;
  }

  for (let i = startIndex; i < this.length; i++) {
    accumulator = callback(accumulator, this[i], i, this);
  }

  return accumulator;
};

const arr = [1, 2, 3, 4];
console.log(arr.customReduce((sum, val) => sum + val, 0)); // 10
```

4.4 Array.prototype.flat

```
Array.prototype.customFlat = function(depth = 1) {
  const result = [];

  for (let i = 0; i < this.length; i++) {
    if (Array.isArray(this[i]) && depth > 0) {
      result.push(...this[i].customFlat(depth - 1));
    } else {
```

```
      result.push(this[i]);
    }
  }

  return result;
};

const arr = [1, [2, [3, [4]]]];
console.log(arr.customFlat(2)); // [1, 2, 3, [4]]
```

5) Object Polyfills

5.1 Object.keys

```
Object.customKeys = function(obj) {
  const keys = [];
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      keys.push(key);
    }
  }
  return keys;
};

const obj = { a: 1, b: 2 };
console.log(Object.customKeys(obj)); // ["a", "b"]
```

5.2 Object.assign

```
Object.customAssign = function(target, ...sources) {
  if (target === null || target === undefined) {
    throw new Error("Cannot convert undefined or null to object");
  }

  const result = Object(target);

  for (const source of sources) {
    if (source === null || source === undefined) continue;

    for (const key in source) {
      if (source.hasOwnProperty(key)) {
        result[key] = source[key];
      }
    }
  }

  return result;
};
```

```
const obj = Object.customAssign({}, { a: 1 }, { b: 2 });
console.log(obj); // { a: 1, b: 2 }
```

6) Function Polyfills

6.1 Function.prototype.bind

```
Function.prototype.customBind = function(thisArg, ...bindArgs) {
  const fn = this;

  return function(...callArgs) {
    return fn.apply(thisArg, [...bindArgs, ...callArgs]);
  };
};

function greet(greeting, name) {
  return `${greeting}, ${name}!`;
}

const boundGreet = greet.customBind(null, "Hello");
console.log(boundGreet("Alice")); // "Hello, Alice!"
```

6.2 Function.prototype.call

```
Function.prototype.customCall = function(thisArg, ...args) {
  const fn = this;
  return fn.apply(thisArg, args);
};
```

6.3 Function.prototype.apply

```
Function.prototype.customApply = function(thisArg, argsArray) {
  const fn = this;
  const context = thisArg || globalThis;
  const key = Symbol();
  context[key] = fn;
  const result = context[key](...(argsArray || []));
  delete context[key];
  return result;
};
```

7) Deep Clone

```
function deepClone(obj, visited = new WeakMap()) {
  if (visited.has(obj)) return visited.get(obj);
```

```
  if (obj === null || typeof obj !== "object") return obj;

  if (obj instanceof Date) return new Date(obj.getTime());
  if (obj instanceof Array) {
    const copy = [];
    visited.set(obj, copy);
    obj.forEach((item, i) => {
      copy[i] = deepClone(item, visited);
    });
    return copy;
  }

  if (obj instanceof Object) {
    const copy = {};
    visited.set(obj, copy);
    for (const key in obj) {
      if (obj.hasOwnProperty(key)) {
        copy[key] = deepClone(obj[key], visited);
      }
    }
    return copy;
  }
}

const original = { a: 1, b: { c: 2 }, d: [3, 4] };
const cloned = deepClone(original);
cloned.b.c = 999;
console.log(original.b.c); // 2 (unchanged)
```

8) Curry Function

```
function curry(fn) {
  const arity = fn.length;

  return function curried(...args) {
    if (args.length >= arity) {
      return fn(...args);
    }
    return (...nextArgs) => curried(...args, ...nextArgs);
  };
}

function add(a, b, c) {
  return a + b + c;
}

const curriedAdd = curry(add);
console.log(curriedAdd(1)(2)(3)); // 6
console.log(curriedAdd(1, 2)(3)); // 6
```

9) Once Function

```
function once(fn) {
  let called = false;
  let result;

  return function(...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}

const init = once(() => {
  console.log("initializing...");
  return "initialized";
});

console.log(init()); // initializing... initialized
console.log(init()); // initialized (no log)
```

10) Retry with Backoff

```
async function retryWithBackoff(fn, maxAttempts = 3, delay = 100) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt === maxAttempts) throw err;

      const backoff = delay * Math.pow(2, attempt - 1);
      await new Promise(r => setTimeout(r, backoff));
    }
  }
}

retryWithBackoff(async () => {
  const res = await fetch("/api/data");
  if (!res.ok) throw new Error("Failed");
  return res.json();
}, 3, 100);
```